AI505 – Optimization

Answers to Obligatory Assignment 1, Spring 2025

Daniele Mensorio

1 Differentiation

1.1 Analytical Differentiation

Analytical expression of derivatives of these functions:

1. $f_1(x) = x^3 - 2x^2 + 3x - 5$

$$f_1^{'}(x) = 3x^2 - 4x + 3$$

2. $f_2(x) = e^x \sin(x)$

$$f_2^{'}(x) = \left(\frac{d}{dx}(e^x)\cdot\sin(x)\right) + \left(e^x \cdot \frac{d}{dx}(\sin(x))\right) = e^x \sin(x) + e^x \cos(x) = e^x\big(\sin(x) + \cos(x)\big)$$

3. $f_3(x) = \frac{1}{1+x^2}$

$$f_3^{'}(x) = \frac{d}{dx}\big((1+x^2)^{-1}\big) = \big(-1\cdot(1+x^2)^{-2}\big)\cdot\frac{d}{dx}(1+x^2) = \big(-(1+x^2)^{-2}\big)\cdot(2x) = -\frac{2x}{(1+x^2)^2}$$

1.2 Numerical Differentiation

The analytical derivatives of the three functions evaluated at $x = 1$ are the following:

- $f_1^{'}(1) = 2$
- $f_2^{'}(1) = e\big(\sin(1) + \cos(1)\big) \approx 3.7560492$
- $f_3^{'}(x) = -\frac{1}{2} = -0.5$

The numerical derivatives, which implement the three methods of finite difference (forward, central and backward difference), applied to the three functions f_1, f_2, and f_3 and evaluated at the point $x = 1$ with a step size $h = 10^{-5}$, give the following results:

1. First function:
   - Forward difference: $f_1^{'}(1) = 2.0000100001027477$;

- Central difference: $f_1'(1) = 2.0000000001019203$;
- Backward difference: $f_1'(1) = 1.9999900001010926$.

2. Second function:

   - Forward difference: $f_2'(1) = 3.7560639140288283$;
   - Central difference: $f_2'(1) = 3.756049227154889$;
   - Backward difference: $f_2'(1) = 3.756034540103314$.

3. Third function:

   - Forward difference: $f_3'(1) = -0.4999975000030687$;
   - Central difference: $f_3'(1) = -0.5000000000032756$;
   - Backward difference: $f_3'(1) = -0.5000024999923802$.

Comparing the results produced by the numerical and analytical differentiation methods (figure 1), it can be seen that, to a small value of h, the numerical differentiation method is affected by the numerical cancellation error, that is, the square root or the cube root of the machine precision for floating point values. So, the size of the steps h balances the precision of the machine for floating point values. The numerical differentiation method is also due to the function's evaluation function to points that are close to each other.

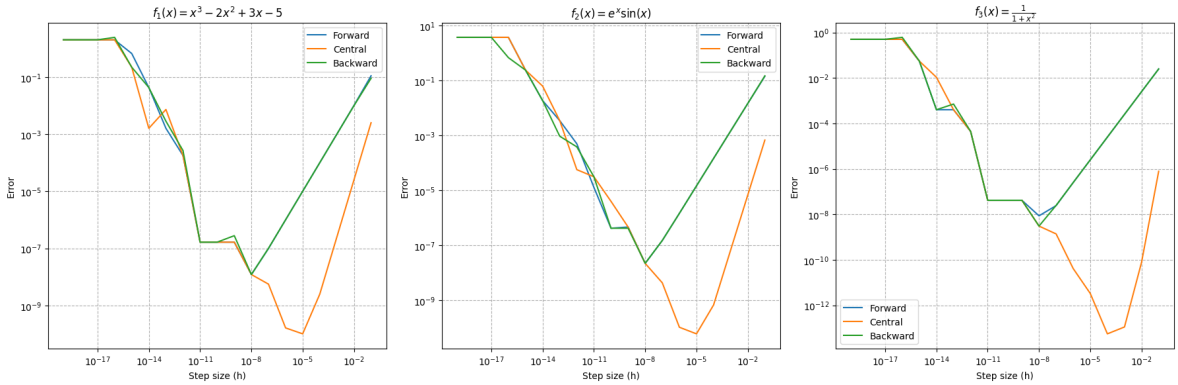
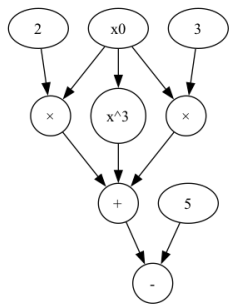


Figure 1: Error comparing numerical and analytical differentiation on step size h

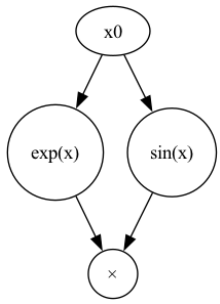
1.3 Automatic Differentiation

Automatic differentiation methods are an application of the chain rule, which can be automated using computational graph.

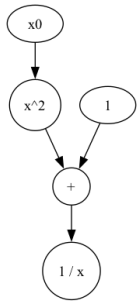
The previous figure (2) is just the theoretical computational graph of the three functions, below it can be seen the same graph evaluated at the point $x_0 = 1$, (figure 3).



(a) $f_1(x) = x^3 - 2x^2 + 3x - 5$

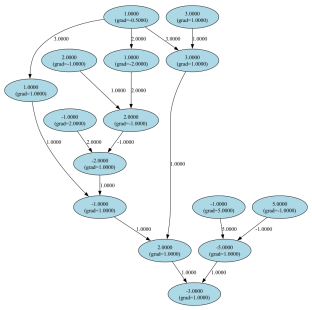


(b) $f_2(x) = e^x \sin(x)$

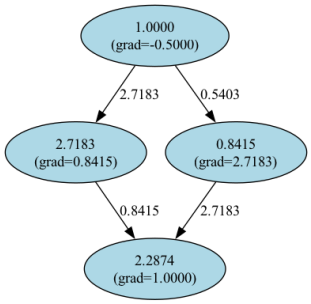


(c) $f_3(x) = \frac{1}{1+x^2}$

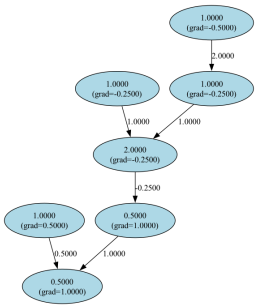
Figure 2: Computational graph of functions.



(a) $f_1(x_0 = 1) = -3$



(b) $f_2(x_0 = 1) = e \cdot \sin(1) \approx 2.2873553$



(c) $f_3(x_0 = 1) = \frac{1}{2} = 0.5$

Figure 3: Computational graph of functions at evaluation point x_0.

The automatic differentiation method applied to the three functions, f_1, f_2, and f_3 and evaluated at point $x_0 = 1$ produces the following:

1. first function: 2.0;
2. second function: 3.7560492270947274;
3. third function: -0.5.

This implies that there is no difference between the automatic and analytical methods, resulting in being more efficient than the numerical one.

Indeed, if we compare the automatic and numerical differentiation, the plot (figure 5) produces the same result, already shown in figure 1.

	Numerical			
	Forward	**Central**	**Backward**	**Automatic**
$f_1(x)$	2.0000100001027477	2.0000000001019203	1.9999900001010926	2.0
$f_2(x)$	3.7560639140288283	3.756049227154889	3.756034540103314	3.7560492270947274
$f_3(x)$	-0.4999975000030687	-0.5000000000032756	-0.5000024999923802	-0.5

Table 1: Analytical and automatic differentiation result of the functions f_1, f_2 and f_3 at evaluation point $x_0 = 1$.

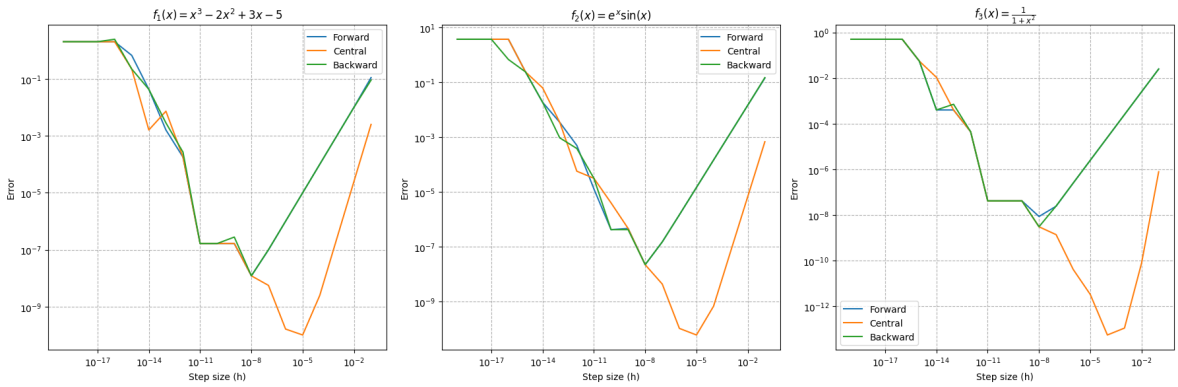


Figure 4: Error comparing automatic and numerical differentiation on step size h

1.4 Multivariate and High-order Differentiation

Considering this function:

$$f_4(x, y) = x^2 y + e^x \sin(y)$$

Its analytical derivative (its gradient) is:

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) = \left(2xy + e^x \sin(y), x^2 + e^x \cos(y) \right)$$

The gradient at evaluation point $x = 3$ and $y = 5$ is:

$$\nabla f(3, 5) = (10.7394911, 14.6975073)$$

In order to compute the automatic differentiation, as we already discussed for one dimensional function, it's good to see its computational graph:

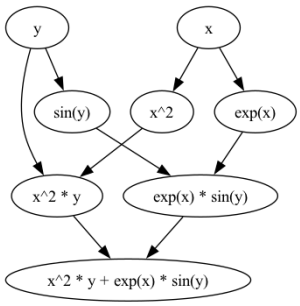


Figure 5: Computational graph of functions $f_4(x, y)$.

The automatic differentiation method gives as result (2):

The error is approximately close to zero, implying the efficiency of this method.

	Automatic Differentiation	**Error**
$\frac{\partial f_4(x,y)}{\partial x}$	10.739491074712578	$2.5287421934194754e-08$
$\frac{\partial f_4(x,y)}{\partial y}$	14.697507299833738	$1.6626167109734524e-10$

Table 2: Analytical and automatic differentiation result of the functions $f_4(x, y)$ at evaluation point $x = 3$ and $y = 5$.

The analytical Hessian (second derivative) of the function $f_4(x, y)$ is:

$$H(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x \partial x} & \frac{\partial f}{\partial y \partial x} \\ \frac{\partial f}{\partial x \partial y} & \frac{\partial f}{\partial y \partial y} \end{bmatrix} = \begin{bmatrix} 2y + e^x \sin(y) & 2x + e^x \cos(y) \\ 2x + e^x \cos(y) & -e^x \sin(y) \end{bmatrix}$$

2 Positive Definite Matrices

2.1 Subtask 2.a

An example of matrix $Q \in \mathbb{R}^{2\times 2}$ that, despite having all positive entries, is not positive definite could be:

$$Q = \begin{bmatrix} 2 & 3 \\ 3 & 1 \end{bmatrix}$$

A matrix is positive definite if $\forall \mathbf{x} \neq 0, \in \mathbb{R}^2$:

$$\mathbf{x}^T Q \mathbf{x} > 0$$

$$\begin{aligned}
&\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} > 0 \\
&\begin{bmatrix} 2x_1 + 3x_2 & 3x_1 + x_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} > 0 \\
&2x_1^2 + 3x_1x_2 + 3x_1x_2 + x_2^2 > 0 \\
&2x_1^2 + x_2^2 + 6x_1x_2 > 0
\end{aligned}$$

So, if we find at least one vector $\mathbf{x} \neq 0, \in \mathbb{R}^2$ that makes the equation not hold, the matrix is not positive definite. if we take the vector $\mathbf{x} = \begin{bmatrix} 1 & -1 \end{bmatrix}$, the equation becomes negative:

$$-3 < 0$$

so, the matrix Q is not positive definite.

2.2 Subtask 2.b

A matrix $A \in \mathbb{R}^{n\times n}$ is positive definite if $\forall \mathbf{x} \neq 0, \in \mathbb{R}^n$:

$$\mathbf{x}^T A \mathbf{x} > 0$$

We know that this is true. So, if we consider the vector $\mathbf{x}$ the basis vector $e_i \in \mathbb{R}^n$ that has 1 in the i-th position and 0 elsewhere:

$$e_i^T A e_i = a_{ii}$$

since all the other therms are multiplied by zeros: the only remaining values are the diagonal values. but we known, for definition of positive definite matrix, that this is always positive:

$$e_i^T A e_i = a_{ii} > 0$$

This is the proof that if a matrix is positive definite, its diagonal values are always positive:

$$a_{ii} < 0 \qquad \forall i$$

2.3 Subtask 2.c

Considering a matrix $A \in \mathbb{R}^{2\times 2}$, not positive definite, the plot of its function:

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x}$$

is, supposing $A = \begin{bmatrix} 2 & 3 \\ 3 & 1 \end{bmatrix}$:

The function $f(\mathbf{x})$ is a quadratic function and since depends also on the matrix A, which is not positive definite, that implies that it does not have a global minimum but it may have saddle points, as can be seen in the center of the plot, where the curvature changes direction.

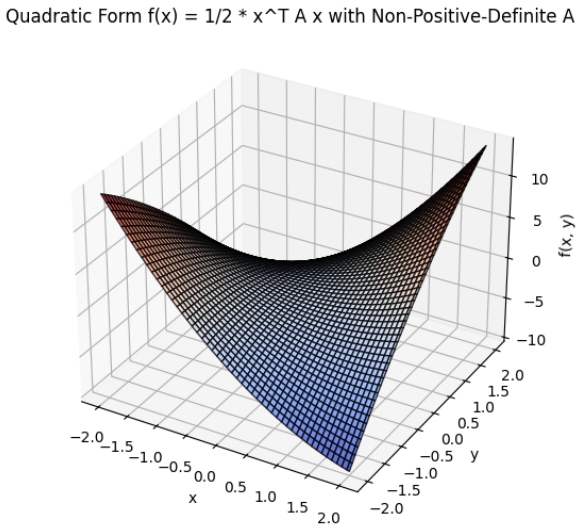


Figure 6: 3D plot of the function $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x}$, with $A = \begin{bmatrix} 2 & 3 \\ 3 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 2}$, not positive definite.

Proof of why matrix A is not positive definite: by definition, if $\exists \mathbf{x} \in \mathbb{R}^2 : \mathbf{x}^T A\mathbf{x} < 0$, the matrix is not positive definite, implying:

$$\begin{aligned}
&\mathbf{x}^T A\mathbf{x} > 0 \\
&\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} > 0 \\
&\begin{bmatrix} 2x_1 + 3x_2 & 3x_1 + x_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} > 0 \\
&2x_1^2 + 3x_1x_2 + 3x_1x_2 + x_2^2 > 0 \\
&2x_1^2 + x_2^2 + 6x_1x_2 > 0
\end{aligned}$$

If we consider $x = \begin{bmatrix} 1 & -1 \end{bmatrix}$, the dis-equation doesn't hold:

$$2 + 1 - 6 = -3 < 0$$

Since the matrix A is not positive definite, it can be concluded that the function $f(\mathbf{x})$ is not convex, but let's go through the proof. A matrix $A \in \mathbb{R}^{2\times 2}$ is convex if $\forall x, y \in \mathbb{R}^2, \forall \alpha \in [0, 1]$:

$$f(\alpha\mathbf{x} + (1-\alpha)\mathbf{y}) \leq \alpha f(\mathbf{x}) + (1-\alpha) f(\mathbf{y})$$

$$\frac{1}{2}(\alpha\mathbf{x}+(1-\alpha)\mathbf{y})^T A(\alpha\mathbf{x}+(1-\alpha)\mathbf{y}) \leq \frac{1}{2}\alpha(\mathbf{x}^T A\mathbf{x}) + \frac{1}{2}(1-\alpha)(\mathbf{y}^T A\mathbf{y})$$
$$(\alpha\mathbf{x}+(1-\alpha)\mathbf{y})^T A(\alpha\mathbf{x}+(1-\alpha)\mathbf{y}) \leq \alpha(\mathbf{x}^T A\mathbf{x}) + (1-\alpha)(\mathbf{y}^T A\mathbf{y})$$
$$(\alpha\mathbf{x}^T+(1-\alpha)\mathbf{y}^T)A(\alpha\mathbf{x}+(1-\alpha)\mathbf{y}) - \alpha(\mathbf{x}^T A\mathbf{x}) - (1-\alpha)(\mathbf{y}^T A\mathbf{y}) \leq 0$$
$$(\alpha(\mathbf{x}^T A)+(1-\alpha)\mathbf{y}^T A)(\alpha\mathbf{x}+(1-\alpha)\mathbf{y}) - \alpha(\mathbf{x}^T A\mathbf{x}) - (1-\alpha)(\mathbf{y}^T A\mathbf{y}) \leq 0$$
$$\alpha^2(\mathbf{x}^T A\mathbf{x}) + \alpha(1-\alpha)(\mathbf{x}^T Ay) + \alpha(1-\alpha)(\mathbf{y}^T Ax) + (1-\alpha)^2(\mathbf{y}^T Ay) - \alpha(\mathbf{x}^T Ax) - (1-\alpha)(\mathbf{y}^T Ay) \leq 0$$
$$\alpha(\alpha-1)(\mathbf{x}^T A\mathbf{x}) + (1-\alpha)(1-\alpha-1)(\mathbf{y}^T A\mathbf{y}) + \alpha(1-\alpha)(\mathbf{x}^T A\mathbf{y} + \mathbf{y}^T Ax) \leq 0$$
$$\alpha(\alpha-1)(\mathbf{x}^T A\mathbf{x}) - \alpha(1-\alpha)(\mathbf{y}^T A\mathbf{y}) + \alpha(1-\alpha)(\mathbf{x}^T A\mathbf{y} + \mathbf{y}^T A\mathbf{x}) \leq 0$$
$$-\alpha(1-\alpha)(\mathbf{x}^T A\mathbf{x}) - \alpha(1-\alpha)(\mathbf{y}^T A\mathbf{y}) + \alpha(1-\alpha)(\mathbf{x}^T A\mathbf{y} + \mathbf{y}^T A\mathbf{x}) \leq 0$$
$$\alpha(1-\alpha)(-\mathbf{x}^T A\mathbf{x} - \mathbf{y}^T A\mathbf{y} + \mathbf{x}^T A\mathbf{y} + \mathbf{y}^T A\mathbf{x}) \leq 0$$
$$\alpha(1-\alpha)[\mathbf{x}^T(A\mathbf{y} - A\mathbf{y}) - \mathbf{y}^T(A\mathbf{y} - A\mathbf{x})] \leq 0$$
$$\alpha(1-\alpha)[(\mathbf{x}^T - \mathbf{y}^T)(A\mathbf{y} - A\mathbf{x})] \leq 0$$
$$\alpha(1-\alpha)[(\mathbf{x}-\mathbf{y})^T A(\mathbf{y}-\mathbf{x})] \leq 0$$
$$-\alpha(1-\alpha)[(\mathbf{x}-\mathbf{y})^T A(\mathbf{x}-\mathbf{y})] \leq 0$$

Now, since $\alpha \in [0,1] \quad \Rightarrow \quad \alpha(1-\alpha) \geq 0$;

But since the matrix A is not positive definite, for definition $(\mathbf{x}-\mathbf{y})^T A(\mathbf{x}-\mathbf{y}) \leq 0$;

so, this means that the equation is not negative and the function is not convex.

What point satisfies $\nabla f(\mathbf{x}) = 0$?

$$\nabla f(\mathbf{x}) = \mathbf{x}^\mathsf{T} A = A\mathbf{x}$$
$$\nabla f(\mathbf{x}) = 0 \quad \Rightarrow \quad \mathbf{x}^\mathsf{T} A = 0$$
$$\begin{bmatrix} 2x_1 + 3x_2 & 3x_1 + x_2 \end{bmatrix} = 0$$

$$\begin{cases} 2x_1 + 3x_2 = 0 \\ 3x_1 + x_2 = 0 \end{cases} \qquad \begin{cases} 2x_1 + 3x_2 = 0 \\ x_2 = -3x_1 \end{cases} \quad \begin{cases} -7x_1 = 0 \\ x_2 = -3x_1 \end{cases} \qquad \begin{cases} x_1 = 0 \\ x_2 = 0 \end{cases}$$

so, the point $\mathbf{x} = \begin{bmatrix} 0 & 0 \end{bmatrix}$ is a stationary point. Now, let's examine if it is a point of minimum, maximum or a saddle one, looking at the Hessian matrix of the function.

$$H(f(\mathbf{x})) = A$$

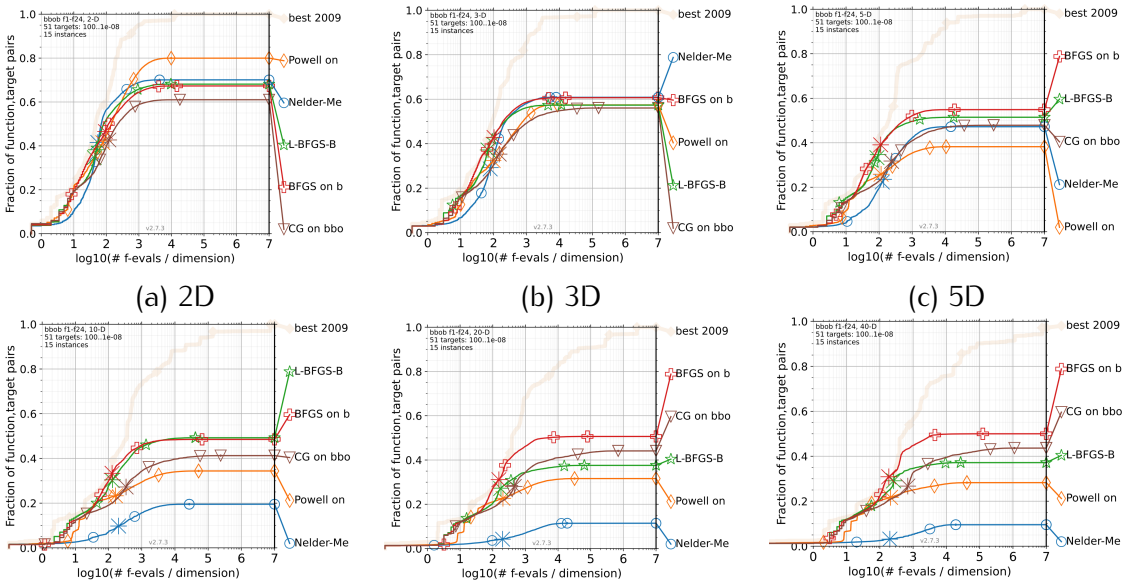
Since the matrix A is not positive definite and the function is not convex, the point $\mathbf{x} = \begin{bmatrix} 0 & 0 \end{bmatrix}$ is a saddle point.

3 Descent Direction Iteration Methods

The algorithm used for comapring result on COCO platform are the following:

- **Nelder-Mead Simplex method**: A derivative-free optimization technique that uses a simplex (a set of $n+1$ points in n dimensions) to explore the search space. The method iteratively reflects, expands, contracts, and shrinks the simplex to move toward a minimum.
- **Powell's Method**: A derivative-free method that performs a series of line searches along a set of conjugate directions. It adjusts these directions as it proceeds to navigate the search space. It is particularly useful when derivatives are unavailable or unreliable, though it might be less efficient on highly non-linear functions.
- **Limited-memory BFGS method**:A quasi-Newton method that approximates the inverse Hessian using limited memory, making it suitable for large-scale problems. It is often used when the full Hessian is too expensive to compute, while still achieving superlinear convergence under favorable conditions.
- **BFGS - Broyden–Fletcher–Goldfarb–Shanno method**: A quasi-Newton optimization algorithm that builds an approximation of the Hessian matrix using gradient evaluations. It requires storing a full Hessian approximation, which can be a drawback in high-dimensional settings.
- **Conjugate gradient method**: An iterative method well-suited for optimization problems, especially those with quadratic or nearly quadratic functions. It avoids the explicit calculation and storage of the Hessian by generating a sequence of conjugate directions along which the objective function is minimized.

Let's see now the runtime profiles over all targets:



(a) 2D (b) 3D (c) 5D

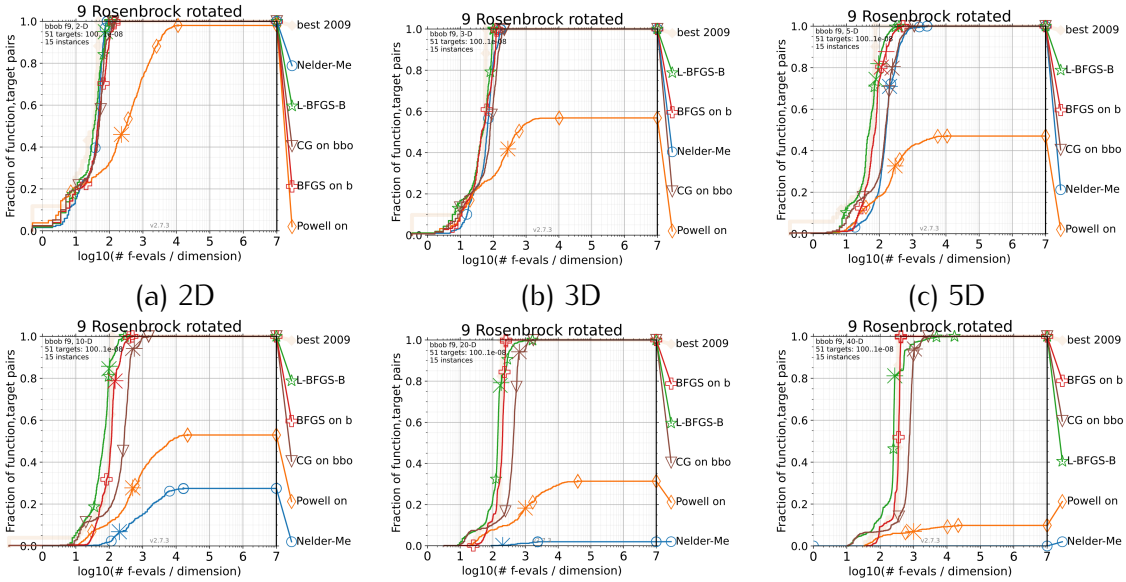
(d) 10D (e) 20D (f) 40D

Figure 7: Runtime profiles over all targets.

These plots (figure 7) compare the performance of different optimization algorithms on a benchmark function (bbob f1-f24) across different dimensions (2D, 3D, 5D, 10D, 20D and 40D). In 2D, Powell performs best, reaching around 0.8, while BFGS, L-BFGS-B, and Nelder-Mead achieve around 0.65-0.7. CG performs the worst. In 3D, BFGS and L-BFGS-B surpass Powell, while Nelder-Mead drops significantly. In 5D, BFGS and L-BFGS-B dominate, while Powell and Nelder-Mead lose effectiveness. CG remains the weakest. ... and so on, we can continue in all the other dimension.

So, we can deduce that derivative-based methods (BFGS, L-BFGS-B) perform well as dimensionality increases: being consistent across dimension. Instead, Powell and Nelder-Mead work well in low dimensions, struggling with higher-dimensional problems.

Now, let's see runtime profiles for some functions: I considered only Rosenbrock original and Rosenbrock rotated.

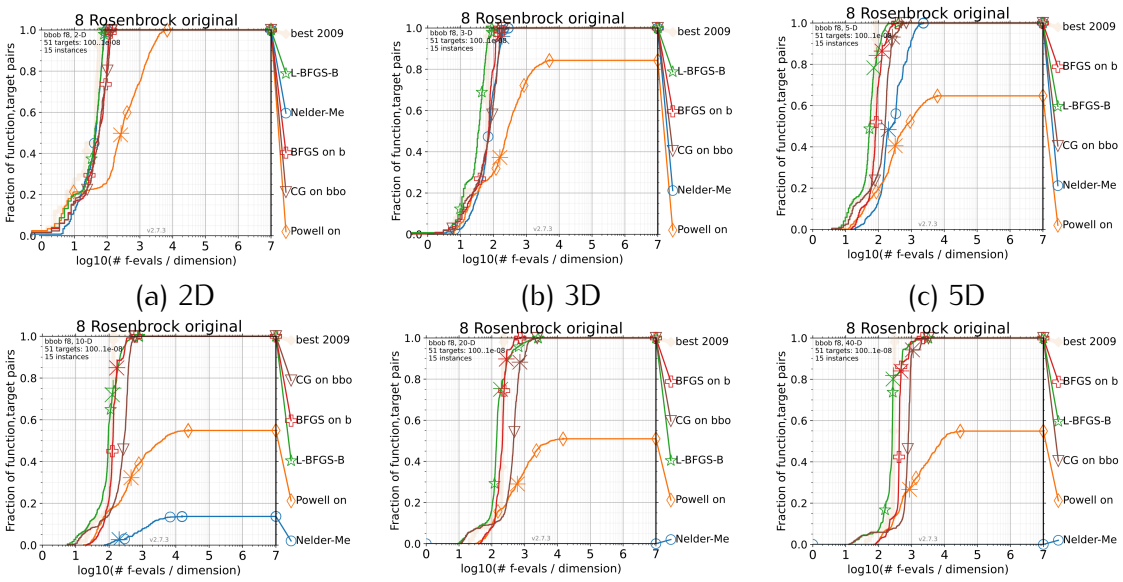


(a) 2D (b) 3D (c) 5D

(d) 10D (e) 20D (f) 40D

Figure 8: Runtime profiles for Rosenbrock rotated function.

These images appear to be performance plots for different dimensions (2D, 3D, 5D, 10D, 20D, 40D) of the Rosenbrock function. The general shape of the curves appears similar across all dimension, showing a sharp increase followed by a plateau. Higher dimensions might take more function evaluations to reach the same performance level. Some algorithms perform significantly better than others in lower dimensions but struggle in higher dimensions.



(a) 2D (b) 3D (c) 5D

(d) 10D (e) 20D (f) 40D

Figure 9: Runtime profiles for Rosenbrock original function.